

Vroomie

Engineering Reference System, Gaps & AI Roadmap

What exists, what is missing, what to improve, and where AI extends the product

Architects · developers · program · database · DevOps
github.com/DGeorgeA/vroomie-app · branch main
Live at vroomie.in

All performance figures herein are measured results on held-out data, reproducible with the committed test harnesses.

1. Stack and topology

Layer	Choice
Frontend	React 18 + Vite, React Router, Zustand (state), TanStack Query
UI	Radix primitives + Tailwind, framer-motion, sonner (toasts)
ML runtime	TensorFlow.js — YAMNet executed on-device , no inference server
Backend	Supabase (Postgres + Auth + Storage + RLS)
Hosting	GitHub Pages, custom domain vroomie.in
PWA	vite-plugin-pwa, Workbox, <code>registerType: 'autoUpdate'</code>
Reporting	jsPDF + jsPDF-autotable (client-side PDF)
Payments	Razorpay via <code>paymentService.js</code>

No server-side compute exists. Every diagnosis runs in the browser. The economic consequence is that marginal cost per diagnosis is zero; the engineering consequence is that model size, memory and main-thread time are hard constraints.

1.1 Codebase size

Area	Files	Lines
<code>src/components</code>	73	9,474
<code>src/pages</code>	17	4,596
<code>src/lib</code> (engine, DSP, reporting)	20	3,199
<code>src/services</code>	5	714
<code>src/contexts</code> / <code>src/store</code>	4	356
<code>scripts</code> (offline factories, QA, harnesses)	39	8,765

Note the ratio: the offline tooling is nearly as large as the application. That is deliberate — the detection thresholds are only defensible because harnesses can re-measure them.

2. Detection pipeline

Two independent matchers over one microphone stream, additive: the fingerprint path never suppresses the embedding path, and a missing fingerprint index degrades to embedding-only rather than failing.

```
getUserMedia { echoCancellation:false, noiseSuppression:false, autoGainControl:false }
  |
  | AudioContext at DEVICE NATIVE RATE (never forced to 16 kHz – iOS Safari)
  | ScriptProcessor, 4096-sample blocks (~85 ms @ 48 kHz)
  |
  |-----|
  | PATH A – FINGERPRINT | PATH B – EMBEDDING |
  | every block, ungated | 1 s window, ~900 ms cadence |
  | resampleBlockTo16k() | resampleTo16k() → exactly 16000 |
  | rolling 5 s ring buffer | silence gate rms < 0.005 |
```

tryMatch() every ~900 ms	rmsNormalize(pcm, 0.05)
constellation + offset coherence	YAMNet → 1024-d + 521 scores
HIT → auto-finalise	domain gate → margin → session rule

Files: `lib/audioFeatureExtractor.js` (capture/routing) · `lib/constellationMatcher.js` (Path A — zero imports, self-contained) · `lib/mlEmbeddingEngine.js` (Path B) · `components/predictive/AudioRecorder.jsx` (session decision) · `scripts/build_constellation_index.mjs` + `scripts/build_reference_fingerprints.mjs` (offline factories).

2.1 Path A — constellation fingerprinting

Constant	Value
NFFT / HOP	1024 (64 ms) / 256 (16 ms)
BANDS	[0,20,40,80,160,320,512]
PEAK_FACTOR	1.6 above band mean
FANOUT, DT_MIN/MAX, DF_MAX	6, 1/48 frames, 160 bins
LISTEN_SECONDS / MIN_LISTEN_SECONDS	5 / 3

Hash packing: $(f1 \& 0x1FF) \ll 15 \mid ((\Delta f + 256) \& 0x1FF) \ll 6 \mid (\Delta t \& 0x3F)$. Matching: each hash votes for $(refId, refTime - queryTime)$; the score is the tallest single-offset bin. A true match aligns the whole recording into one bin; unrelated audio scatters.

```
normalised = score / min(queryHashes, ref.hash_count)
instant    : score ≥ 600 AND normalised ≥ 0.050
sustained  : score ≥ 480 AND normalised ≥ 0.042 AND same ref on 2 consecutive attempts
```

Index (`public/constellation_v1.json`): 8 references, 155,288 entries, 1.6 MB, two parallel base64 `Int32Arrays` hydrated into `Map<hash, Int32Array>` via `subarray` views. Exclusions: a synthetic sine tone; the misfire sample (acoustically an irregular idle — caused the only two healthy false fires measured); anything under `MIN_REF_SECONDS = 4.0`.

2.2 Path B — YAMNet embedding

```
ANOMALY_THRESHOLD 0.45  ANCHOR_MARGIN 0.04  NEAR_ANCHOR_MARGIN 0.02
VEHICLE_SCORE_FLOOR 0.03  GENERIC_INTERFERER_CEILING 0.15
WEAK_INTERFERER_VEHICLE_FLOOR 0.02  WEAK_INTERFERER_CEILING 0.30
marginToConfidence = clamp(0.70 + 1.08 × (margin - 0.04), 0.70, 0.97)
```

Domain gate (521 YAMNet classes, 53 vehicle/mechanical, interferers = human + music ranges + TV/Radio/Silence/Whistling):

```
accept = vehicleTop1
  | (!interfererTop1 && interfererScore < 0.15)
  | (vehicleScore ≥ 0.03 && vehicleScore > interfererScore)
  | (interfererTop1 && vehicleScore ≥ 0.02 && interfererScore ≤ 0.30) ← speaker rescue
```

Anchors are load-bearing. A healthy idling car scores 0.892–0.918 cosine against power-steering references — higher than several genuine faults. 78 healthy + 16 interferer anchors supply the "compared to what?".

Session decision (`AudioRecorder.jsx`): primary ≥45% of accepted windows on one *family*, min 3 accepted; recovery vote at ≥60% total candidates with a 1.10× similarity-sum tie-break; near tier at ≥85% for the 65–69% "possible" band.

2.3 Reference artifact

`public/fingerprints_v9.json` — 1024-d, int8 quantised (`value = min + byte × scale`), **352 fault embeddings + 94 anchors**.

Families: alternator 100, power_steering 100, then 24 each for intake_leak, misfire, piston_knock, rocker_valve, serpentine_belt, timing_chain; motor_starter 8. Per-family cap = 100 (before it, power_steering held 58.8% and absorbed 8 of 9 healthy false alarms).

Eight augmentation variants: `orig`, `band`, `noise`, `rate+`, `rate-`, `echo`, `speaker`, `phonespk`.

3. Data layer

3.1 Tables in use

Table	Purpose
<code>analyses</code>	Every diagnostic session, incl. aborted (<code>status: 'rejected'</code>)
<code>user_roles</code>	UUID-keyed RBAC; <code>is_admin()</code> SECURITY DEFINER
<code>app_features</code>	Global feature flags (ethanol check), admin-write via RLS
<code>subscriber_base</code>	Subscription/consent records
<code>subscriptions</code>	Razorpay subscription state
<code>user_preferences</code>	Per-user settings
<code>customer_feedback</code>	Post-session feedback
<code>anomaly_references</code> / <code>anomaly_embeddings</code>	Legacy pgvector tables (<code>ivfflat</code> , <code>vector_cosine_ops</code>) — superseded by the static artifact; not read at runtime

`analyses` columns: `id`, `created_at` (server default, immutable), `vehicle_id`, `audio_file_url`, `duration_seconds`, `status`, `confidence_score`, `anomalies_detected` jsonb, `analysis_result` jsonb, `detection_mode`, `detection_source`, `ml_confidence`, `signal_similarity`, `final_decision`, `processed_at`, `notes`.

Storage: bucket `anomaly-patterns` (reference WAVs, public read; writes require dashboard/service key).

3.2 RLS posture

Correctly locked: `user_roles` (read-own, admin-read-all), `app_features` (read-all, admin-write only), `subscriber_base` (own rows).

`analyses` is **public-read and public-insert** — see Part II, finding **SEC-1**.

4. Build and deploy

`.github/workflows/deploy.yml` — `ubuntu-latest`, Node 20, `npm install --legacy-peer-deps`, `npm run build:production`, publish to Pages. Triggered on push to `main`.

PWA: `registerType: 'autoUpdate'`, Workbox precache glob covers

`js,css,html,ico,png,svg,jpg,jpeg,woff2,wasm,json` with `maximumFileSizeToCacheInBytes: 5 MB` — which is why both artifacts (1.6 MB + ~700 KB) are cached offline.

The pipeline has no quality gate. Build only: no lint, no unit tests, no QA suite, no boot check. Consequence is documented in Part II, **OPS-1**.

5. Observability

Every session — success *and* failure — writes diagnostics into

`analyses.analysis_result.session_diagnostics`:

```
windows_analyzed, accepted, rejected, rejected_silence, rejected_domain,
domain_heard_as[], capture_settings{sampleRate, autoGainControl,
echoCancellation, noiseSuppression}, best_fingerprint{label,score,normalized},
engine_build
```

`capture_settings` records the **applied** constraints, not the requested ones — some Android builds silently ignore `noiseSuppression:false`.

This telemetry diagnosed the 13 August demo failure two days later without device access: fingerprint scores 79/167/88 versus 549–736 for identical audio, with 37 of 42 windows still accepted, isolating Zoom's acoustic echo cancellation.

6. Test and measurement assets

Harness	Answers
<code>stress_detection_matrix.mjs</code>	8 refs × 4 channels × 4 volumes × 3 offsets → 384/384
<code>verify_sustained_tier.mjs</code>	Fingerprint false fires: 0/140 healthy, 0/9 interferers
<code>rca_healthy_fp_sweep.mjs</code>	Embedding healthy FP: 6/140 (4.3%)
<code>benchmark_discrimination.mjs</code> + <code>rule_explorer.mjs</code>	Held-out matrix; instant rule grid search
<code>qa_unit_tests.mjs</code> , <code>qa_ethanol_feature.mjs</code>	Drift-proof constant/policy assertions
<code>review_constellation_shipped.mjs</code>	Re-gate thresholds after any index change

PART II — GAP ANALYSIS

Severity: **P0** ship-blocking / security · **P1** significant · **P2** worth doing.

7. What we have

- A **measured**, two-path detection engine with documented operating points and reproducible harnesses.
- **Zero-marginal-cost inference** — fully on-device.
- **Self-diagnosing failures** — every abort explains itself and persists remotely readable evidence.
- **Correct RBAC** on roles and feature flags: UUID-keyed, RLS-enforced, fails closed, no email comparison in app code.
- **Deterministic reference builds** — offline factory with QC, augmentation, quantisation; index and query share the same hashing function by import, so they cannot drift.

- **PWA with offline model caching** and one-command deploy.
- **Honest UX**: graded confidence, explicit "possible — verification required" tier, disclaimers, no fabricated healthy verdicts.

8. What we do not have

SEC-1 · analyses **is world-readable and world-writable — P0**

The MVP policy "**Public read access**" plus public insert means **any holder of the anon key (which ships in the client bundle) can read every diagnostic session from every user, and write arbitrary rows**. Sessions also carry no `user_id`, so ownership cannot be enforced without a schema change. Before any real user base: add `user_id uuid references auth.users`, backfill, and replace with owner-scoped policies. Retain a service-role path for aggregate analytics.

SEC-2 · Audio uploads are publicly readable — P1

`audio_file_url` points at public storage. Engine-bay recordings may capture bystander speech. Needs signed URLs and a retention policy.

OPS-1 · CI has no quality gate — P0

The pipeline builds and deploys. It does not lint, run tests, run the QA suites, or verify the bundle boots. A temporal-dead-zone error once compiled cleanly and **shipped a blank white site to production**. Minimum viable gate: `eslint` → `qa_unit_tests` → `qa_ethanol_feature` → headless boot assertion (`#root` non-empty) → deploy.

OPS-2 · No staging environment — P1

`build:staging` exists but nothing consumes it. Every change goes straight to the production domain.

OPS-3 · No error/perf monitoring — P1

No Sentry or equivalent. Client exceptions are invisible unless they happen to surface in a session row.

OPS-4 · `--legacy-peer-deps` **masks a dependency conflict — P2**

Unresolved peer ranges will eventually break a clean install. No lockfile audit or Dependabot.

QA-1 · No automated test runner — P1

`vitest` is a devDependency with no test suite. The `.mjs` harnesses are excellent but are manually invoked, slow (minutes), and network-dependent (they fetch from Supabase and TF Hub). No fast unit layer over the pure DSP functions, which are trivially testable.

QA-2 · No physical-device validation — P1

All acoustic figures come from channel simulations plus in-app injection and limited field telemetry. No matrix across handset models, OS versions or microphone hardware. This is the single largest gap between measured and real-world performance.

DATA-1 · Reference library is the binding constraint — P1

Nine fault families. Piston, serpentine, rocker, timing, intake each have **one** source recording. Injector tick and exhaust leak have none and cannot be detected. Both headline accuracy numbers (4.3% healthy false alarms, ~2-in-3 unseen recall) are library-bound, not algorithm-bound.

DATA-2 · Legacy pgvector tables are dead weight — P2

`anomaly_references` / `anomaly_embeddings` with ivfflat indexes are unused at runtime. Either remove, or repurpose deliberately (see AI-4).

ARCH-1 · Audio DSP runs on the main thread — P1

`ScriptProcessor` is deprecated; `AudioWorklet` is the supported path. FFT and matching execute on the UI thread — measured 22 ms per match, comfortable today, but it competes with React rendering and will degrade on low-end hardware.

ARCH-2 · No versioned artifact contract — P2

`constellation_v1.json` and `fingerprints_v9.json` are fetched by fixed path. A client with a stale service worker can pair new code with an old artifact. There is no compatibility assertion at load.

PM-1 · No analytics or funnel instrumentation — P1

Diagnostic telemetry exists; product telemetry does not. Activation, retention, conversion and detection-outcome distribution are all unmeasured.

PM-2 · No feedback→label loop — P1

`customer_feedback` exists but is not wired to detections. The mechanism that would turn usage into training data is absent — which is the stated moat.

9. What can be done better

#	Improvement	Why	Effort
1	<code>user_id</code> + owner-scoped RLS on <code>analyses</code>	Closes SEC-1	S
2	CI quality gate incl. headless boot check	Prevents shipping a dead site	S
3	Move DSP to <code>AudioWorklet</code>	Removes main-thread contention	M
4	Vitest layer over pure DSP (FFT, resample, hashing, normalisation)	Seconds-fast regression net	S
5	Artifact version handshake	Prevents stale-SW mismatch	S
6	Sentry + Web Vitals	Client failures become visible	S
7	Staging on a preview domain	Real pre-prod validation	S

#	Improvement	Why	Effort
8	Collect real fault recordings, phone-captured, multiple vehicles per family	The only fix for DATA-1	Ongoing
9	Golden-file fixtures for engine outputs	Detects silent numerical drift	M
10	Retention + signed URLs for audio	Closes SEC-2	S
11	Device matrix testing (BrowserStack/real handsets)	Closes QA-2	M
12	Split the 9.5k-line component tree; extract a features/ layer	Maintainability at 73 components	L

PART III — WHERE AI EXTENDS THIS

The current engine uses AI in exactly one place: YAMNet as a fixed, pre-trained feature extractor. Everything downstream is deterministic arithmetic. That is a deliberate and correct starting point — it is debuggable and it forced the discipline that makes the system trustworthy. The opportunities below are ordered by expected value per unit of risk.

AI-1 · Fine-tune the embedding space on vehicle audio — highest value

YAMNet is trained on general AudioSet: dogs, doorbells, rain. Its embedding is not optimised to separate *alternator whine* from *belt squeal*. A **metric-learning head** (triplet or supervised-contrastive) trained on top of the frozen YAMNet embedding, using labelled fault vs healthy audio, would directly widen the margin the whole system depends on.

- Small model (a 1024→128 projection), trains in minutes on modest GPU, ships as a few hundred KB of extra weights.
- Runs client-side alongside the existing artifact; the margin rule and anchors are unchanged.
- **Directly attacks both headline weaknesses:** better separation lowers the 4.3% false-alarm rate *and* raises the ~2-in-3 recall.
- Prerequisite: labelled data (DATA-1). This is the concrete engineering reason the raise is for data + GPU.

AI-2 · Learned rejection instead of hand-tuned gates

The domain gate is four hand-written clauses, each added after a specific failure. A small classifier — vehicle vs not-vehicle, trained on the accumulated accepted/rejected windows already logged in telemetry — would replace heuristics with something that improves as data arrives. Keep the hand-written gate as a fallback and shadow-run the classifier until it demonstrably beats it.

AI-3 · Synthetic augmentation of the reference library

Rather than only hand-coded channel simulations (band, echo, speaker), use **learned room-impulse-response convolution** and generative audio augmentation to expand each reference into a far

richer set of realistic captures. Directly addresses "one recording per family" without waiting for field collection. Must be validated against the same false-positive gates — augmentation that widens a family's territory without adding information is exactly the failure measured when piston/serpentine "enrichment" was reverted.

AI-4 · Per-vehicle baselining (repurpose the pgvector tables)

Record each user's own healthy engine at onboarding, embed it, and store it as a **personal anchor**. Detection then asks *"has this car changed?"* rather than *"does this resemble a generic fault?"* — the strongest possible discriminator, because it eliminates inter-vehicle variance entirely. The dormant `anomaly_references` table with its ivfflat index is the natural home. This is likely the single largest accuracy step available.

AI-5 · Active learning from workshop outcomes

Close the loop in PM-2: when a workshop confirms or refutes a detection, that becomes a label. Prioritise for review the sessions where the engine was *least* certain (near the margin boundary) — active learning extracts the most information per label. This is the mechanism that makes GoMechanic's service volume compound into an accuracy moat.

AI-6 · LLM for the report narrative, not the diagnosis

Fault→advice text is currently a static dictionary. An LLM could generate vehicle-specific, plain-language explanations and workshop instructions. **Constraint that must not be relaxed:** the LLM must never influence the detection or the confidence figure — it renders a decision the deterministic engine already made. Generate server-side or at build time to avoid shipping keys; cache aggressively.

AI-7 · Anomaly detection for the unknown-fault case

Every fault family requires reference audio, so unknown faults are invisible. A one-class/autoencoder model trained purely on **healthy** engine audio could flag "this doesn't sound like any healthy engine" without knowing what the fault is — expanding coverage beyond the nine labelled families and turning the healthy-audio corpus (far easier to collect) into an asset.

Sequencing

1. **Collect labelled data** (DATA-1) — nothing else unlocks without it
2. **AI-4 per-vehicle baselining** — largest accuracy gain, uses existing infrastructure
3. **AI-1 embedding fine-tune** — direct attack on both weaknesses
4. **AI-5 active learning** — makes the moat compound
5. AI-3 / AI-2 / AI-7 — incremental
6. AI-6 — product polish, no accuracy impact

One rule for all of the above, learned repeatedly and expensively in this codebase: every change is gated on the measured false-positive suites before it ships. Sensitivity is easy; sensitivity without false alarms is the product.

Reproduce every figure here from the committed harnesses. Companion documents:

`VROOMIE_PIPELINE_TECHNICAL.md` (*pipeline depth*), `VROOMIE_HOW_IT_LISTENS.md` (*non-technical*),
`VROOMIE_SESSION_HANDOFF.md` (*state and lessons*).